

NumPy for Data Science

Day 2 — Lecture Notes & Practical Tasks

Intermediate to Applied · Hands-On Exercises Included

1. Advanced Boolean & Fancy Indexing

Building on Day 1 basics, this section covers compound conditions, `np.where` with arrays, and fancy indexing — all essential for real data manipulation.

1.1 Compound Boolean Conditions

```
import numpy as np

data = np.array([15, 42, -3, 87, -12, 55, 0, 33])

# AND / OR / NOT (use & | ~ not 'and'/'or'/'not')
pos_small = data[(data > 0) & (data < 50)]
print(pos_small)          # [15 42 33]

neg_or_big = data[(data < 0) | (data > 80)]
print(neg_or_big)        # [-3 87 -12]

not_zero = data[~(data == 0)]
print(not_zero)           # removes 0

# np.any / np.all
print(np.any(data < 0))   # True
print(np.all(data > 0))   # False
print(np.all(data > 0, axis=0)) # per column if 2D

# np.nonzero — returns index tuple
idx = np.nonzero(data > 40)
print(idx)                # (array([1, 3, 5]),)
print(data[idx])          # [42 87 55]
```

1.2 `np.where` — Vectorised if-else

```
import numpy as np

scores = np.array([45, 82, 67, 55, 91, 38, 74])

# Simple replacement
clipped = np.where(scores >= 60, scores, 60)

# String labels
labels = np.where(scores >= 80, 'Dist',
                  np.where(scores >= 60, 'Merit', 'Pass'))
print(labels)

# Compute on condition
bonus = np.where(scores >= 80, scores * 1.1, scores)
print(bonus.astype(int))

# np.select - multiple conditions
conds = [scores >= 80, scores >= 60, scores >= 40]
vals = ['A', 'B', 'C']
grades = np.select(conds, vals, default='D')
print(grades)
```

1.3 Fancy Indexing & np.ix_

```
import numpy as np

m = np.arange(25).reshape(5, 5)

# Select specific rows
print(m[[0, 2, 4], :])

# Select specific (row, col) pairs (diagonal-like)
r = [0, 1, 2]; c = [4, 3, 2]
print(m[r, c])          # [4, 8, 12]

# np.ix_ - all combinations (cross product)
rows = [0, 2]; cols = [1, 3]
sub = m[np.ix_(rows, cols)]
print(sub)              # 2x2 sub-matrix

# Reorder rows/cols
print(m[::-1, :])       # flip rows
print(m[:, ::-1])       # flip cols
```

1.4 Tasks — Advanced Indexing

Task 1.4.1: Given a 6×6 matrix of random integers 1–100, extract all elements that are divisible by both 3 and 5.

Task 1.4.2: Use `np.where` to replace all values below the mean of an array with 0 and above with 1 (binarise).

Task 1.4.3: Use `np.ix_` to extract a 3×3 sub-matrix from rows [1,3,4] and columns [0,2,4] of a 5×5 array.

Task 1.4.4: Sort a 4×4 random matrix row-wise and column-wise separately (without loops).

Task 1.4.5: Use `np.select` to classify 15 random scores into: 'A'(≥85), 'B'(≥70), 'C'(≥55), 'D'(≥40), 'F'(below 40).

2. Broadcasting — Deep Dive

Broadcasting allows NumPy to perform operations on arrays of different shapes without copying data, following three rules.

2.1 Broadcasting Rules

- Rule 1: If arrays have different ndim, the shape of the smaller is padded with 1s on the LEFT.
- Rule 2: Arrays with size 1 along any dimension are stretched to match the other.
- Rule 3: If sizes differ and neither is 1 → `ValueError`.

```
import numpy as np

# (3,1) + (4,) → both become (3,4)
col = np.array([[0],[10],[20]]) # shape (3,1)
row = np.array([1, 2, 3, 4])    # shape (4,)
print(col + row)
# [[ 1  2  3  4]
#  [11 12 13 14]
#  [21 22 23 24]]

# Normalise rows: subtract each row's mean
data = np.random.randint(1,100,(4,5)).astype(float)
row_means = data.mean(axis=1, keepdims=True) # (4,1)
centred = data - row_means
print(centred.mean(axis=1)) # ~all zeros

# Outer product using broadcasting
a = np.array([1,2,3])[ :, np.newaxis] # (3,1)
b = np.array([10,20,30,40])           # (4,)
print(a * b)                          # (3,4)
```

2.2 Tasks — Broadcasting

Task 2.2.1: Without loops, add [1, 2, 3, 4] to every row of a (5,4) matrix and subtract [10,20,30,40,50] from every column.

Task 2.2.2: Normalise each column of a 6x4 random matrix to have mean=0 and std=1 using broadcasting.

Task 2.2.3: Build a distance matrix: given 5 points on a number line, create a 5x5 matrix where entry [i,j] = |points[i] - points[j]|. Use broadcasting, no loops.

3. Linear Algebra with np.linalg

3.1 Matrix Multiplication

```
import numpy as np

A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

# Three equivalent ways
print(np.dot(A, B))
print(A @ B)                # recommended (Python 3.5+)
print(np.matmul(A, B))

# Element-wise vs matrix multiply
print(A * B)                # element-wise
print(A @ B)                # matrix multiply
```

3.2 linalg Functions

```
import numpy as np

A = np.array([[2.,1.],[1.,3.]])

# Inverse
Ai = np.linalg.inv(A)
print(A @ Ai)                # should be identity

# Determinant
print(np.linalg.det(A))      # 5.0

# Eigenvalues & eigenvectors
vals, vecs = np.linalg.eig(A)
print(vals)                  # eigenvalues
print(vecs)                  # columns are eigenvectors

# Solve linear system Ax = b
```

```
b = np.array([8.,13.])
x = np.linalg.solve(A, b)
print(x)                    # solution vector
print(A @ x)                # verify == b

# Norms
v = np.array([3., 4.])
print(np.linalg.norm(v))    # 5.0 (L2 norm)
print(np.linalg.norm(A, 'fro')) # Frobenius
```

3.3 Tasks — Linear Algebra

Task 3.3.1: Create a random 3×3 matrix A. Verify $A @ \text{inv}(A) \approx \text{identity}$ using `np.allclose`.

Task 3.3.2: Solve the system: $2x+y=5$, $x+3y=10$ using `np.linalg.solve`. Verify your answer.

Task 3.3.3: Generate a symmetric matrix $S = A.T @ A$. Compute its eigenvalues and verify they are all non-negative.

Task 3.3.4: Compute the dot product of two random unit vectors (normalise with `np.linalg.norm` first). Interpret the result.

4. Sorting, Searching & Set Operations

4.1 Sorting

```
import numpy as np

arr = np.array([3,1,4,1,5,9,2,6,5])

print(np.sort(arr))          # sorted copy
arr.sort()                   # in-place sort
print(np.argsort(arr))       # indices that sort arr

# 2D sort
m = np.random.randint(1,20, (3,4))
print(np.sort(m, axis=1))    # sort each row
print(np.sort(m, axis=0))    # sort each col

# Top-k (largest 3)
k = 3
top_idx = np.argsort(arr)[-k:][::-1]
print(arr[top_idx])
```

4.2 Searching & Set Ops

```
import numpy as np

arr = np.array([1,1,2,3,4,4,5])

vals, counts = np.unique(arr, return_counts=True)
print(vals, counts)

# searchsorted - binary search
print(np.searchsorted(arr, 3))    # index to insert 3

a = np.array([1,2,3,4,5])
b = np.array([3,4,5,6,7])
print(np.intersect1d(a,b))      # [3 4 5]
print(np.union1d(a,b))          # [1 2 3 4 5 6 7]
print(np.setdiff1d(a,b))        # [1 2]
print(np.in1d(a,b))             # [F F T T T]
```

4.3 Tasks — Sorting & Searching

Task 4.3.1: Given a 1D array of 20 random integers, find the indices of the 5 smallest and 5 largest values without fully sorting.

Task 4.3.2: Sort a 5×5 matrix so each row is sorted ascending and each column is sorted ascending (hint: sort rows first, then columns).

Task 4.3.3: From two random integer arrays of size 15 (range 1–20), find common elements, elements only in A, and elements only in B.

5. Working with Real Data

5.1 Loading & Saving

```
import numpy as np

# Save / load binary (fastest)
arr = np.arange(100).reshape(10,10)
np.save('data.npy', arr)
loaded = np.load('data.npy')

# Multiple arrays
np.savez('multi.npz', arr1=arr, arr2=arr*2)
f = np.load('multi.npz')
print(f['arr1'].shape, f['arr2'].shape)

# CSV text format
np.savetxt('out.csv', arr, delimiter=',', fmt='%d')
```

```
back = np.loadtxt('out.csv', delimiter=',', dtype=int)

# genfromtxt handles missing values
data = np.genfromtxt('messy.csv',
                    delimiter=',',
                    filling_values=0,
                    skip_header=1)
```

5.2 Column Operations on Loaded Data

```
import numpy as np

# Simulate: cols = id, math, science, english
data = np.column_stack([
    np.arange(1,11),
    np.random.randint(40,100, (10,3))
]).astype(float)

ids      = data[:,0]
math     = data[:,1]
science  = data[:,2]
english  = data[:,3]

avg      = data[:,1:].mean(axis=1)
total    = data[:,1:].sum(axis=1)
rank     = total.argsort()[::-1].argsort() + 1

best_sub = np.argmax(data[:,1:].mean(axis=0))
print("Best subject index:", best_sub)
```

5.3 Tasks — File I/O & Data

Task 5.3.1: Create a 20×5 random float array, save it as both .npy and .csv, reload both, and confirm they are equal using `np.allclose`.

Task 5.3.2: Simulate a dataset of 50 students with 4 subject marks. Save to CSV. Reload and compute: class topper, subject-wise averages, and students scoring below 50 in any subject.

6. Performance & Vectorisation

6.1 Vectorisation Patterns

```
import numpy as np, time

n = 3_000_000
a = np.random.rand(n)
b = np.random.rand(n)
```

```
# BAD - Python loop
t = time.perf_counter()
c = [a[i]+b[i] for i in range(n)]
print(f"Loop: {time.perf_counter()-t:.3f}s")

# GOOD - vectorised
t = time.perf_counter()
c = a + b
print(f"NumPy: {time.perf_counter()-t:.4f}s")

# In-place ops save memory allocation
a += b          # same memory as a
np.add(a, b, out=a) # explicit out

# Pre-allocation
result = np.empty(n)
np.multiply(a, b, out=result)
```

6.2 einsum & Advanced

```
import numpy as np

A = np.random.rand(100,200)
B = np.random.rand(200,150)

# Matrix multiply via einsum
C = np.einsum('ij,jk->ik', A, B) # same as A@B

# Trace (sum of diagonal)
M = np.random.rand(5,5)
print(np.einsum('ii->', M))      # same as np.trace(M)

# Row-wise dot products
a = np.random.rand(10,4)
b = np.random.rand(10,4)
print(np.einsum('ij,ij->i', a, b)) # 10 dot products
```

6.3 Tasks — Performance

Task 6.3.1: Rewrite the following loop using vectorisation: `result = []; for i in range(len(x)): result.append(x[i]**2 + 2*x[i] + 1)`. Benchmark both.

Task 6.3.2: Use `np.einsum` to compute the outer product of two vectors and verify it matches `np.outer`.

Task 6.3.3: Generate a float32 and float64 array of 1M elements. Compare memory usage and computation speed for squaring each.

7. Comprehensive Challenge Tasks

Integrate everything from both days. Each challenge should be solved using NumPy only — no pandas, no loops unless explicitly stated.

7.1 Mini Statistics Engine

Write a function `stats_report(arr)` that accepts a 2D array and prints: shape, min, max, mean, median, std, 25th and 75th percentile, and the row and column indices of the global maximum.

7.2 Image as Array

Simulate a grayscale image as a random integer array of shape (100, 100) with values 0–255. (a) Flip it horizontally and vertically. (b) Crop the centre 40×40 region. (c) Increase brightness by 20 (clip at 255 using `np.clip`). (d) Threshold: pixels > 128 → 255, rest → 0.

7.3 Student Ranking System

Given a 30×5 array of student marks (subjects), compute: (a) overall average per student, (b) rank each student (1=best), (c) find the top 5 students and bottom 5 students, (d) compute the correlation between subject 1 and subject 2 scores using `np.corrcoef`.

7.4 Monte Carlo Pi Estimation

Estimate π using Monte Carlo: generate $N=1,000,000$ random (x,y) points in $[0,1] \times [0,1]$. Count how many fall inside the unit circle ($x^2+y^2 \leq 1$). $\pi \approx 4 \times (\text{inside}/\text{total})$. Do this entirely with NumPy — no Python loops.

7.5 Portfolio Risk Analysis

Create a (12, 5) array representing monthly returns of 5 assets over 12 months. Compute: (a) mean return per asset, (b) covariance matrix using `np.cov`, (c) portfolio variance for equal weights $[0.2]*5$, (d) identify the highest-risk and lowest-risk asset.